CCA EXAMINATION MANAGEMENT SYSTEM

Comprehensive System Architecture, Algorithmic Engine & Capstone Defense Dossier

Author / Candidate: Capstone Engineering Team | Institution: City College of Angeles | Cloud Target: 165.22.100.18 (cca-exam.duckdns.org)

Executive Technical Abstract: This dossier documents the architectural foundations and engineering decisions of the CCA Exam Management System (EXAM_MANAGEMENT). Developed for academic institutional operations, it encompasses a defense-in-depth security perimeter, robust session management, MySQL connection pooling, a deterministic Constraint Satisfaction Problem (CSP) scheduling engine with tiered relaxation, and an enterprise deployment stack on DigitalOcean utilizing Nginx, Gunicorn, Systemd, and DuckDNS.

1. System Overview & Architectural Topology

The system adheres to a decoupled 5-tier web architecture engineered for high concurrency, deterministic security, zero timetable conflicts, and high availability:

Layer	Component & Technology	Core Architectural Responsibilities
1. Client Layer	Desktop & Mobile Browsers (HTML5, Vanilla JS, Bootstrap, CSS)	Delivers responsive role-based interfaces; includes CSRF tokens on all mutating requests; polls unread notifications via an asynchronous ticker without resetting session idle timers.
2. Edge & DNS	DuckDNS Dynamic DNS & Nginx 1.18+ (Reverse Proxy)	Binds cca-exam.duckdns.org to droplet IP (165.22.100.18). Terminates SSL/TLS. Enforces 50MB request buffering and a 300s proxy timeout to allow uninterrupted batch processing.
3. WSGI Server	Gunicorn 21.2.x (Pre-fork Worker Model)	Hosts 3 dedicated worker processes on loopback (127.0.0.1:9999) under Linux Systemd supervisor (cca_exam.service) with automatic crash revival.
4. App & Logic	Flask 3.1.x Framework (app.py, logic.py, scheduler.py)	Implements ProxyFix middleware, Flask-Limiter, Bcrypt hashing, NIST password checks, RBAC route guards, rolling 20-min session expiry, and the CSP scheduling engine.
5. Persistence	MySQL 8.x InnoDB (database.py, Connection Pool)	Managed by MySQLConnectionPool (size: 10, pool_reset_session=True) with graceful fallback to direct connections; autonomous schema updates and audit logging.

2. End-to-End User Authentication & Session Security

Authentication is protected by defense-in-depth measures against brute-forcing, session fixation, and CSRF:

- **Step 1: Request Intake & Rate Limiting:** POST requests to /login are throttled by Flask-Limiter at 15 req/min per IP. Attacks are stopped immediately with HTTP 429.
- **Step 2: Cryptographic Password Verification:** verify_login() queries the user via parameterized SQL. Passwords are verified against stored hashes using bcrypt.checkpw() (Blowfish salt, cost factor 12).
- **Step 3: Account State Verification:** If is_active = 0, authentication is blocked with 'inactive', keeping unapproved registrations locked out.
- **Step 4: Session Cleansing (Anti-Fixation):** When credentials pass, session.clear() wipes pre-existing session cookies, neutralizing session fixation vectors.
- **Step 5: Session Hydration:** Injects user_id, role, instructor_id, section, institute_id, and float epoch _last_active.
- **Step 6: Role Redirection:** _redirect_by_role() routes the user: Admin → /dashboard, Proctor → /proctor/dashboard, Secretary → /secretary/dashboard, Student → /student/timetable.

2.1 Detailed Walkthrough of the 4 User Roles

Access control is enforced strictly across all four institutional user classes:

Role	Access Controls & Data Scoping	Primary Capabilities & System Flows
Student	Guarded by require_role('student'). Data scoped strictly to the student's assigned section and institute_id.	Read-only access to published timetables (gated by is_schedule_published()). Official PDF timetable export. Irregular students access the **Irregular Student Portal** to verify cross-section schedules and review subject conflicts.
Proctor (Faculty)	Guarded by require_role('proctor'). Linked via session['instructor_id'] to instructor master records.	Timetable hidden until published. Submits time availability windows (proctor_availability) used as hard constraints by the scheduler. Inspects assigned rooms and monitors cumulative duty hours against institutional caps.
Secretary	Guarded by require_role('secretary', 'admin'). Departmentally scoped by session['institute_id'].	Monitors test paper custody via exam_collection (Pending → Collected → Returned). Tracks proctor attendance, records absences (proctor_absence), and executes one-click dispatch of standby invigilators from the relievers pool.
Administrator & Super Admin	Guarded by require_role('admin') and require_super_admin(). Global access across all institutes.	Full institutional governance: batch imports curriculums, rooms, and proctors; executes automated CSP scheduler; adjusts dates/rooms with live conflict checking; manages dual-authorization schedule reset requests; manages user credentials.

Rolling Inactivity Watchdog (20-Minute Inactivity Timeout):

To comply with NIST SP 800-63B and institutional data protection guidelines, an application-level watchdog @app.before_request check_session_inactivity() executes on every incoming request. If time.time() - session['_last_active'] > 1200 seconds (20 minutes), the session is cleared, an AUTH_SESSION_TIMEOUT event is written to audit_logs, and the user is redirected to /login. Polling endpoints (such as /api/notifications/unread-count) are specifically exempt from refreshing _last_active, ensuring background JavaScript tickers do not keep unattended sessions active.

3. Backend & Database Architecture (app.py & database.py)

The backend service layer is engineered around connection resilience, thread safety, and strict separation of concerns:

MySQL Connection Pooling & Burst Fallback:

Opening raw database connections for every request causes high socket latency and risks database connection exhaustion under peak student traffic. In system/database.py, we implement a persistent connection pool via mysql.connector.pooling.MySQLConnectionPool initialized with pool_size = 10 and pool_reset_session = True. When connections return to the pool via db.close(), all user variables, active transactions, and temporary states are cleared automatically. If extreme traffic spikes exhaust the 10 pooled connections, open_db() captures the pool exception and immediately spawns a direct database connection, ensuring requests never fail with 500 server errors.

Autonomous Database Schema Synchronization:

To eliminate migration errors when deploying across environments, ensure_database_schema() runs automatically upon application launch. It dynamically inspects existing table definitions and non-destructively adds required structures, including audit_logs, exam_archives, and schedule_reset_requests, as well as essential operational columns (schedule_published, max_daily_proctor_hrs).

Non-Repudiation Audit Logging:

All state-changing actions—such as timetable modifications, user promotions, schedule resets, and student approvals—are permanently recorded in the audit_logs table with the actor's user_id, user_role, client ip_address, action type, and detailed description. This provides forensic accountability and full compliance with institutional security standards.

4. The Scheduling Engine (scheduler.py & Constraint Satisfaction)

Examination scheduling is mathematically categorized as the **Examination Timetabling Problem (ETP)**, a proven **NP-Hard Combinatorial Optimization Problem**. A brute-force search over all combinations of exams, rooms, dates, and proctors is computationally intractable. To generate a 100% conflict-free timetable in seconds, we engineered a deterministic **Constraint Satisfaction Problem (CSP) Heuristic Engine with a Tiered Relaxation Ladder**.

Hard Constraints (Mathematically Invariant - Zero Violations Allowed):

- **1. Zero Room Double-Booking:** Ensured by interval collision checks room_cal[r_id] against candidate [start, end]. No two exams ever occupy the same physical room at overlapping times.
- **2. Zero Proctor Double-Booking:** Ensured by inst_cal[i_id]. A proctor can never be scheduled for two concurrent examinations.
- **3. Zero Student Overlap + 1-Hour Rest Buffer:** section_free(sec, s, e) checks interval [s - 1 hour, e + 1 hour] against section_cal. Students never have simultaneous exams, and a mandatory 1-hour rest window is guaranteed between exams to eliminate cognitive fatigue.
- **4. Sister-Section Synchronization:** Exams are pre-grouped by content_group_id or subject_id. All class sections enrolled in the same course take the exam at the exact same time slot across parallel rooms. This completely eliminates test question leaks.
- **5. Strict Anti-Self-Proctoring:** Faculty members are strictly barred from proctoring examinations for subjects they teach. Pre-tokenized subjects_taught sets are matched against the exam group tokens before candidate evaluation.
- **6. Proctor Availability Windows:** O(1) dictionary lookup against proctor_availability ensures instructors are only assigned during time windows they officially submitted.
- **7. PWD Accessibility Matching:** Faculty members with physical disabilities (is_pwd = 1) are strictly assigned to rooms marked as wheelchair-accessible (is_pwd_accessible = 1) on ground floors or elevator-serviced wings.
- **8. Seating Capacity Bounds:** The engine verifies that room.capacity >= section_student_count (or institute default capacity).

Soft Constraints & The 4-Tier Relaxation Ladder:

In dense exam periods, rigid daily proctor limits can cause artificial scheduling deadlocks. The engine resolves this by dynamically evaluating candidate slots across a progressive 4-tier relaxation ladder:

Tier	Daily Major Cap per Section	Daily Proctoring Cap per Faculty	Operational Purpose
Tier 1	Max 2 Major Exams	Max 4.0 Hours	Ideal institutional balance; standard target load.
Tier 2	Max 2 Major Exams	Max 5.0 Hours	Absorbs minor bottlenecks while preserving student study balance.
Tier 3	Max 3 Major Exams	Max 6.5 Hours	High-density periods with limited faculty inventory.
Tier 4	Max 3 Major Exams	Max 8.0 Hours (Absolute Ceiling)	Emergency fail-safe; ensures 100% placement rather than unassigned exams.

Fair Rotation & Campus Travel Heuristics:

Candidate proctors are sorted using a multi-attribute comparator: (1) fewest proctoring hours worked on that day, (2) fewest cumulative hours worked across the exam period, and (3) a round-robin assignment counter. This guarantees equitable workload distribution. Additionally, when a proctor moves between different buildings, the engine enforces a mandatory 15-minute inter-building travel buffer (travel_buffer_min = 15).

5. Deployment & Cloud Infrastructure

The production system is hosted on an enterprise Linux Virtual Private Server (VPS) on DigitalOcean, architected for high availability and automated recovery:

- **DigitalOcean Ubuntu Droplet (165.22.100.18):** Provides high-performance compute and isolated memory, operating in the Asia/Manila (+08:00) timezone to match institutional schedules.
- **DuckDNS Dynamic DNS (cca-exam.duckdns.org):** Binds the droplet's public IP to a persistent domain name, enabling automated SSL/TLS certificate renewal via Let's Encrypt Certbot.
- **Nginx Reverse Proxy & SSL Gateway:** Listens on ports 80 and 443. Terminates SSL/TLS; forwards authenticated traffic to Gunicorn on 127.0.0.1:9999. Configured with client_max_body_size 50M for batch file uploads and proxy_read_timeout 300s to handle long-running batch scheduling without 504 timeouts.
- **Gunicorn WSGI Application Server:** Runs 3 pre-forked worker processes. If one worker process is computing a complex schedule, the remaining workers continue to serve student schedule inquiries without degradation.
- **Linux Systemd Supervisor (cca_exam.service):** Controls the Gunicorn process daemon with Restart=always. If the process encounters a fatal exception or the host reboots, Systemd revives the service within milliseconds. Environment variables (DB credentials, FLASK_SECRET_KEY) are securely isolated in the service definition.
- **1-Click Automated Deployment Pipeline:** Executed via update_server_now.bat and deploy_remote.sh. The script packages source files into cca_exam_deploy.zip, transfers them via SCP, purges stale .pyc bytecode, verifies database settings, and issues a zero-downtime reload to Gunicorn and Nginx.

6. Capstone Defense: Panelist Anticipated Questions & Answers

Be prepared to deliver these concise, professional engineering responses during your defense presentation:

Q1: Why did you develop a custom heuristic CSP engine instead of using an off-the-shelf solver like Google OR-Tools CP-SAT directly in production?

Answer: While commercial solvers like OR-Tools are powerful general-purpose tools, they introduce heavy C++ binary dependencies, high RAM overhead, and deployment fragility on lean VPS instances. Academic scheduling includes unique institutional rules—such as sister-section synchronization, 1-hour cognitive rest buffers, anti-self-proctoring token matching, building transit buffers, and PWD room accessibility. Our custom CSP heuristic with a Tiered Relaxation Ladder is specifically tailored to these requirements, executing in under two seconds with 100% deterministic constraint satisfaction.

Q2: How does the system mathematically prevent test question leaks between different sections?

Answer: We enforce Sister-Section Synchronization in system/scheduler.py. All exams sharing the same content_group_id or subject_id are treated as an atomic scheduling cluster. The engine locks a single synchronized timestamp for the entire cluster. All sister sections take the exam at the exact same hour and date across parallel rooms, eliminating any opportunity for questions to be leaked between sections.

Q3: What mechanisms prevent session hijacking and cross-site scripting (XSS) cookie theft?

Answer: We implement four layers of session security: (1) 256-bit entropy cryptographic cookie signing via Flask's secret key; (2) SESSION_COOKIE_HTTPONLY = True, making cookies completely inaccessible to client-side JavaScript; (3) SESSION_COOKIE_SAMESITE = 'Lax' with Flask-WTF CSRF tokens on all mutating requests; and (4) a rolling 20-minute inactivity watchdog that purges expired sessions automatically.

Q4: What happens if all 10 pooled MySQL connections are busy when a burst of students logs in?

Answer: In system/database.py, open_db() catches connection pool exhaustion exceptions gracefully. Instead of dropping the user's request with an HTTP 500 Internal Server Error, the system immediately spawns an on-demand direct MySQL connection to handle the burst, returning to the pool once traffic normalizes.

Q5: How does your system prevent proctor burnout during grueling multi-day exam periods?

Answer: Proctor assignment uses a three-tier fair rotation heuristic: (1) fewest hours worked on the scheduled date, (2) fewest total cumulative hours across the entire exam period, and (3) a round-robin assignment tiebreaker. In addition, consecutive proctoring is capped at 3 hours, daily hours are strictly constrained by the active relaxation tier, and a 15-minute inter-building travel buffer is enforced.